

HDOC Day-27

Write a C program that displays the following menu and uses a switch statement to print the “Pyramid/Diamond pattern” selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameterless function with no return value. Note: For each activity, prepare the algorithm and test-case table. Then write, compile and execute the C program, and test it against the prepared test cases.

1. Full Pyramid

Algorithm

1. Start.
2. Define the function fullPyramid().
3. Read the number of rows n.
4. Initialize i = 1.
5. Repeat for every row from 1 through n.
6. Print n-i spaces.
7. Print 2*i-1 stars.
8. Move to the next line.
9. Continue until i = n.
10. Stop.

2. Inverted Pyramid

Algorithm

1. Start.
2. Define the function invertedPyramid().
3. Read the number of rows n.
4. Initialize i = n.
5. Print n-i spaces.
6. Print 2*i-1 stars.
7. Move to the next line.
8. Decrease i by 1.
9. Continue until i = 1.
10. Stop.

3. Diamond Pattern

Algorithm

1. Start.
2. Define the function `diamondPattern()`.
3. Read the number of rows n for the upper half.
4. Initialize $i = 1$.
5. Print $n-i$ spaces.
6. Print $2*i-1$ stars.
7. Continue until $i = n$.
8. Set $i = n-1$.
9. Print $n-i$ spaces.
10. Print $2*i-1$ stars.
11. Decrease i by 1.
12. Continue until $i = 1$.
13. Stop.

4. Palindromic Number Pyramid

Algorithm

1. Start.
2. Define the function `palindromicNumberPyramid()`.
3. Read the number of rows n .
4. Initialize $i = 1$.
5. Print $n-i$ pairs of spaces.
6. Print numbers from 1 to i .
7. Print numbers from $i-1$ down to 1.
8. Move to the next line.
9. Increase i by 1.
10. Continue until $i = n$.
11. Stop.

5. Repeated-Number Pyramid

Algorithm

1. Start.
2. Define the function `repeatedNumberPyramid()`.
3. Read the number of rows `n`.
4. Initialize `i = 1`.
5. Print `n-i` pairs of spaces.
6. Print the number `i`, $2*i-1$ times.
7. Move to the next line.
8. Increase `i` by 1.
9. Continue until `i = n`.
10. Stop.

C Program

```
#include <stdio.h>

// Function declarations
void fullPyramid();
void invertedPyramid();
void diamondPattern();
void palindromicNumberPyramid();
void repeatedNumberPyramid();

int main()
{
    int choice;
    do
    {
        printf("\n===== PYRAMID / DIAMOND PATTERNS =====\n");
        printf("1. Full Pyramid\n");
        printf("2. Inverted Pyramid\n");
        printf("3. Diamond Pattern\n");
        printf("4. Palindromic Number Pyramid\n");
        printf("5. Repeated-Number Pyramid\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1: fullPyramid(); break;
            case 2: invertedPyramid(); break;
            case 3: diamondPattern(); break;
            case 4: palindromicNumberPyramid(); break;
            case 5: repeatedNumberPyramid(); break;
            case 6: printf("Program exited successfully.\n"); break;
            default: printf("Invalid choice! Please enter 1 to 6.\n");
        }
    } while(choice != 6);
    return 0;
}

// 1. Full Pyramid
void fullPyramid()
{
    int n, i, j;
    printf("\nEnter the number of rows: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        // Print spaces
        for(j = 1; j <= n - i; j++)
            printf(" ");
        // Print stars
        for(j = 1; j <= 2 * i - 1; j++)
            printf("* ");
    }
}
```

```

        printf("\n");
    }
}

// 2. Inverted Pyramid
void invertedPyramid()
{
    int n, i, j;
    printf("\nEnter the number of rows: ");
    scanf("%d", &n);
    for(i = n; i >= 1; i--)
    {
        // Print spaces
        for(j = 1; j <= n - i; j++)
            printf(" ");
        // Print stars
        for(j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

// 3. Diamond Pattern
void diamondPattern()
{
    int n, i, j;
    printf("\nEnter the number of rows for upper half: ");
    scanf("%d", &n);
    // Upper half including middle row
    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= n - i; j++) printf(" ");
        for(j = 1; j <= 2 * i - 1; j++) printf("* ");
        printf("\n");
    }
    // Lower half
    for(i = n - 1; i >= 1; i--)
    {
        for(j = 1; j <= n - i; j++) printf(" ");
        for(j = 1; j <= 2 * i - 1; j++) printf("* ");
        printf("\n");
    }
}

// 4. Palindromic Number Pyramid
void palindromicNumberPyramid()
{
    int n, i, j;
    printf("\nEnter the number of rows: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {

```

```

        // Print spaces
        for(j = 1; j <= n - i; j++) printf(" ");
        // Increasing numbers
        for(j = 1; j <= i; j++) printf("%d ", j);
        // Decreasing numbers
        for(j = i - 1; j >= 1; j--) printf("%d ", j);
        printf("\n");
    }
}

// 5. Repeated-Number Pyramid
void repeatedNumberPyramid()
{
    int n, i, j;
    printf("\nEnter the number of rows: ");
    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        // Print spaces
        for(j = 1; j <= n - i; j++) printf(" ");
        // Print current row number (2i - 1) times
        for(j = 1; j <= 2 * i - 1; j++) printf("%d ", i);
        printf("\n");
    }
}

```

Test Case Table

Test Case	Choice	Input	Expected Result
1	1	4	Full pyramid displayed
2	2	4	Inverted pyramid displayed
3	3	4	Diamond pattern displayed
4	5	4	Repeated-Number pyramid displayed
5	6	-	Program exits successfully
6	8	-	Invalid choice! displayed

Compilation and Execution

Test Case 1:

```
(kali@kali)-[~]  
$ ./a.out
```

———— PYRAMID / DIAMOND PATTERNS ————

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 1

Enter the number of rows: 4

```
  *  
 * * *  
* * * * *  
* * * * * *
```

Test Case 2:

--- PYRAMID / DIAMOND PATTERNS

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 2

Enter the number of rows: 4

```
* * * * *
 * * * *
  * * *
   *
```

Test Case 3:

———— PYRAMID / DIAMOND PATTERNS ————

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 3

Enter the number of rows for upper half: 4

```
  *
 * * *
* * * * *
* * * * * * *
 * * * *
  * * *
   *
```

Test Case 4:

————— PYRAMID / DIAMOND PATTERNS —————

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 5

Enter the number of rows: 4

1

222

33333

44444444

Test Case 5:

```
(kali@kali)-[~]  
$ ./a.out
```

————— PYRAMID / DIAMOND PATTERNS —————

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 6

Program exited successfully.

Test Case 6:

```
(kali㉿kali)-[~]  
$ ./a.out
```

PYRAMID / DIAMOND PATTERNS

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 8

Invalid choice! Please enter 1 to 6.